ANALISIS Y DISEÑO DE ALGORITMOS

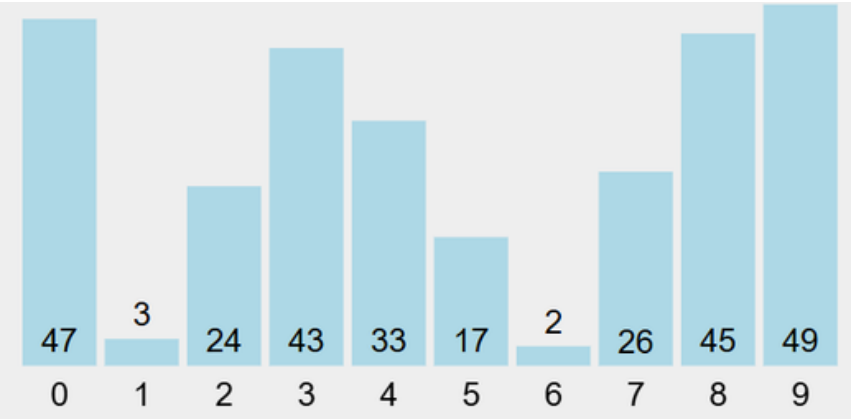
Proyecto Algoritmos de Ordenamiento

ASAF NEFTALY SOSA RAMÍREZ

Bubble Sort

Se ejecutó Bubble Sort sobre el arreglo personalizado **A = 3,44,38,5,47,15,36,26,45,49** con **N = 10**. La implementación detectó parada temprana (no hubo swaps en la pasada 5), por lo que el algoritmo terminó antes de completar todas las pasadas teóricas.

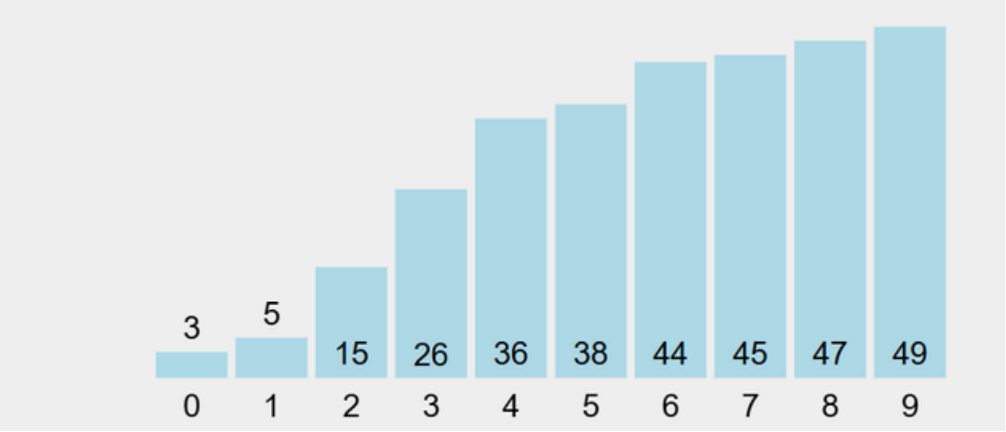
Datos de Entrada y configuración



Resultados de la corrida

Métrica	Valor
Comparaciones totales	35
Swaps totales	14
Pasadas realizadas	5
Tiempo	44 segundos

Ordenamiento Final



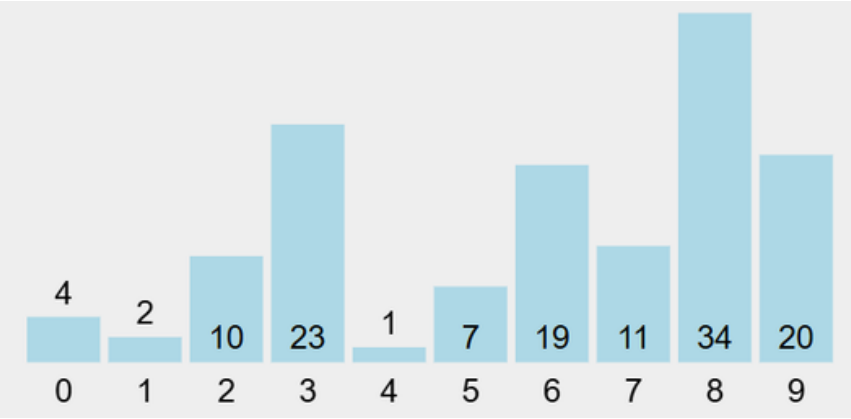
Comparación

- Método simple que compara y troca adyacentes repetidamente.
- Cómo se comparaFrente a Selection: más swaps pero puede detectar orden temprano; Selection hace menos swaps pero siempre recorre todo.
- Frente a Insertion: peor en entradas parcialmente ordenadas; Insertion suele ser mucho más eficiente cuando hay orden parcial.
- Frente a Merge/Quick/Radix: claramente inferior en escalabilidad; solo útil para enseñanza o N muy pequeño.
- Recomendación práctica
- Utilizarlo solo para aprendizaje o cuando la simplicidad extrema importa; evita en benchmarks reales.

Selection Sort

Ejecución de Insertion Sort sobre N = 10 con A = 4, 2, 10, 23, 1, 7, 19, 11, 34, 20. La implementación clásica realiza N-1 pasadas; aquí se observaron 19 comparaciones, 12 desplazamientos (shifts) y 9 pasadas, con el arreglo final ordenado 1, 2, 4, 7, 10, 11, 19, 20, 23, 34.

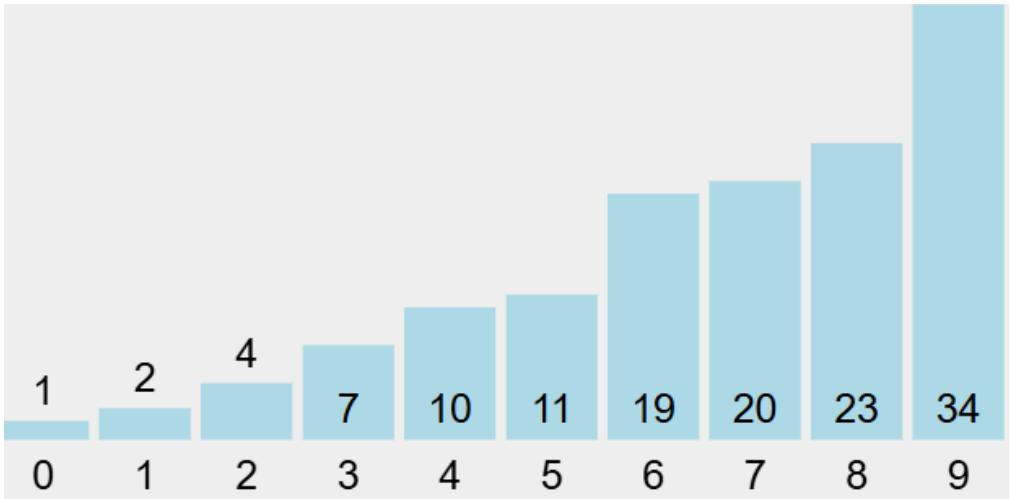
Datos de Entrada y configuración



Resultados de la corrida

Métrica	Valor
Comparaciones totales	19
Desplazamientos	12
Pasadas realizadas	9
Tiempo	1 minuto 3 segundos

Ordenamiento Final



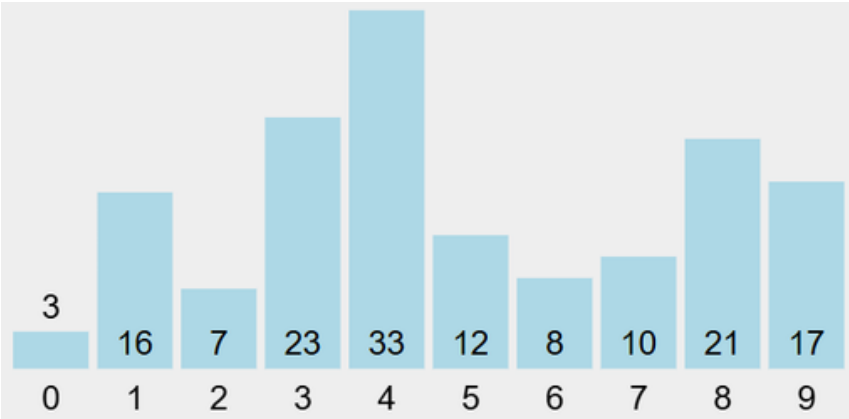
Comparación

- Selecciona el mínimo y lo coloca en su posición; pocas escrituras.
- Cómo se comparaFrente a Bubble: menos swaps, mismo orden temporal; más predecible.
- Frente a Insertion: peor cuando hay orden parcial; Insertion hace menos comparaciones en esos casos.
- Frente a Merge/Quick/Radix: peor en tiempo; ventaja solo cuando las escrituras son costosas (memoria flash).
- Recomendación práctica
- Útil cuando escribir en memoria es caro y N pequeño; de lo contrario, elegir otro algoritmo.

Insertion Sort

Ejecución de Insertion Sort sobre **N = 10 con A = 3, 16, 7, 23, 33, 12, 8, 10, 21, 17**. La implementación clásica realizó 26 comparaciones, 17 desplazamientos (shifts) y 9 pasadas, produciendo el arreglo ordenado 3, 7, 8, 10, 12, 16, 17, 21, 23, 33.

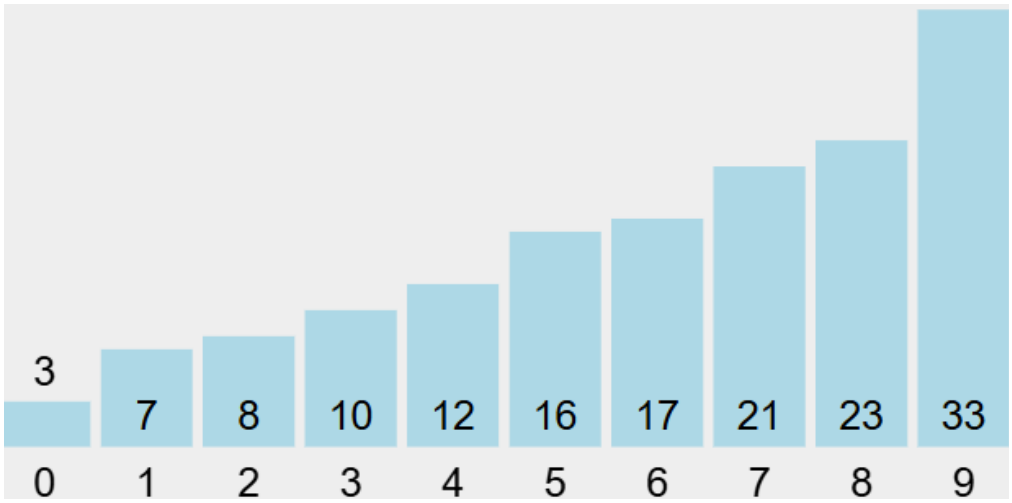
Datos de Entrada y configuración



Resultados de la corrida

Métrica	Valor
Comparaciones totales	26
Desplazamientos	17
Pasadas realizadas	9
Tiempo	23 segundos

Ordenamiento Final



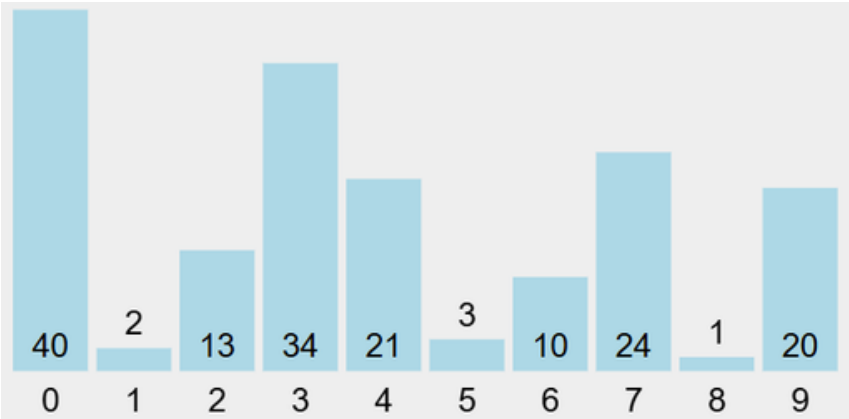
Comparación

- Inserta cada elemento en el prefijo ya ordenado; muy eficiente si el arreglo está casi ordenado.
- Cómo se comparaFrente a Bubble/Selection: generalmente mejor en comparaciones y movimientos para entradas reales pequeñas.
- Frente a Merge: para N pequeño o casi ordenado, Insertion suele ser más rápido por menor overhead; Merge gana en entradas grandes y desordenadas.
- Frente a Quick: Quick suele superar a Insertion en entradas grandes; Insertion es competitivo en arreglos pequeños y se usa dentro de Quick/Merge para subarreglos pequeños.
- Frente a Radix: Radix vence cuando hay muchos elementos y dígitos limitados; Insertion es preferible para N pequeño o cuando los elementos no son enteros.

Merge Sort

Se ejecutó Merge Sort sobre **N = 10 con A = 40, 2, 13, 34, 21, 3, 10, 24, 1, 20**. La ejecución realiza divisiones recursivas y fusiones (merges). En esta corrida se registraron 24 comparaciones durante las fusiones, 9 operaciones de fusión (merges) y 34 escrituras de elementos en los arreglos temporales durante las fusiones.

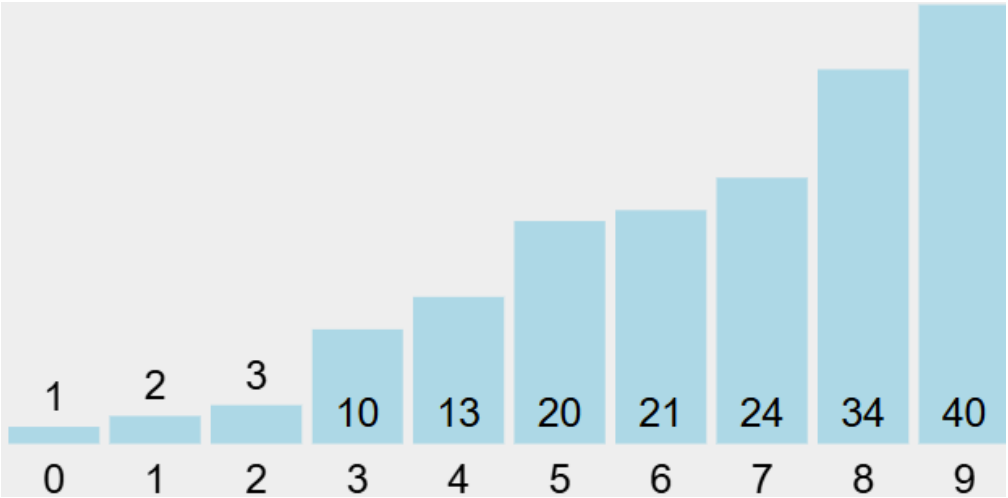
Datos de Entrada y configuración



Resultados de la corrida

Métrica	Valor
Comparaciones totales	24
Número de merges realizados	9
Escrituras en arreglos temporales	34
Tiempo	2 segundos

Ordenamiento Final



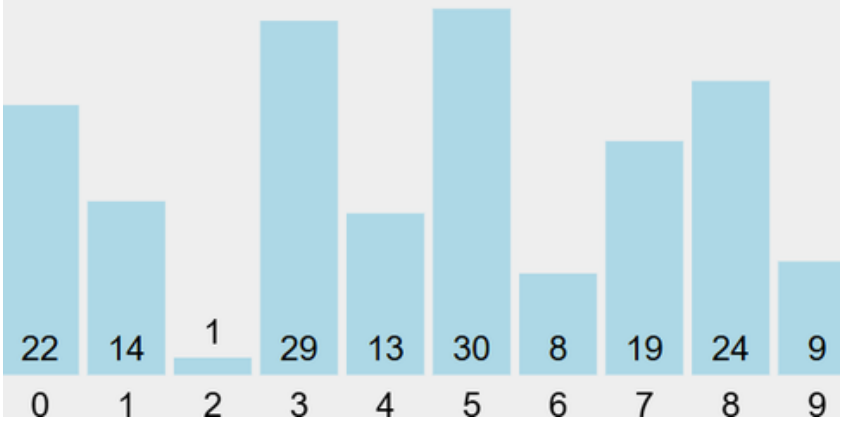
Comparación

- Divide y fusiona; coste O(n\log n) garantizado y estable.
- Cómo se compara Frente a Quick: Merge es estable y predecible; Quick suele ser más rápido en promedio y usa menos memoria extra en implementaciones in-place, pero Quick tiene peor caso cuadrático.
- Frente a Insertion/Bubble/Selection: mucho mejor para N grandes; el overhead de copias hace que en N muy pequeño Insertion pueda ganar.
- Frente a Radix: Radix puede ser más rápido para enteros con pocos dígitos; Merge es más general y funciona bien con comparadores arbitrarios.
- Recomendación práctica
- Eligir Merge cuando necesitemos estabilidad y rendimiento garantizado, o cuando trabajemos con datos que no son enteros.

Quick Sort

Ejecución de Quick Sort sobre N = 10 con **A = 22, 14, 1, 29, 13, 30, 8, 19, 24, 9**. Asumo la implementación clásica con partición Lomuto (pivote = último elemento del segmento) y que contamos comparaciones como cada evaluación A[j] <= pivot y swaps como intercambios efectivos entre posiciones (no contamos "self-swaps" donde el índice es el mismo).

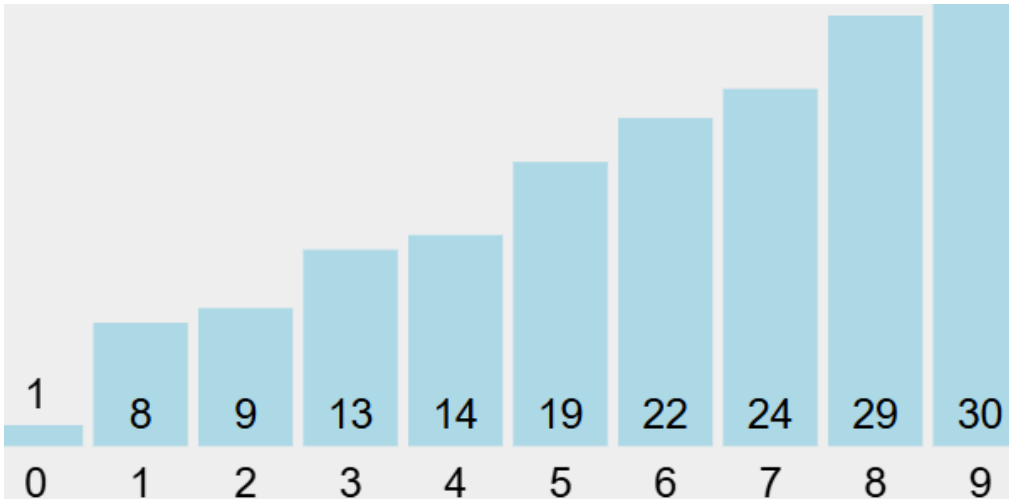
Datos de Entrada y configuración



Resultados de la corrida

Métrica	Valor
Comparaciones totales	21
Swaps totales	9
Particiones / llamadas con partición	6
Tiempo	55 segundos

Ordenamiento Final



Comparación

- Particiona alrededor de un pivote; excelente rendimiento promedio y uso de memoria eficiente.
- Cómo se compara Frente a Merge: suele ser más rápido en memoria por menos copias; Merge gana en estabilidad y en peor caso garantizado.
- Frente a Insertion: Quick domina en N grandes; Insertion domina en subarreglos pequeños y casi ordenados. Por eso Quick suele combinarse con Insertion para subproblemas pequeños.
- Frente a Radix: Radix puede superar a Quick en enteros grandes con pocos dígitos; Quick es más general para comparables.
- Recomendación práctica
- Buena elección por defecto para ordenación en memoria; usar pivoteo aleatorio o mediana para evitar peor caso.

Radix Sort

Ejecución de Radix Sort (LSD, base 10) sobre **N = 10 con A = 39, 22, 16, 14, 12, 7, 30, 28, 3, 10**. El algoritmo procesa dígito a dígito (menos significativo a más significativo). Se realizaron 2 pasadas (d = 2 dígitos), no hay comparaciones entre elementos (usa buckets), y el arreglo final ordenado es 3, 7, 10, 12, 14, 16, 22, 28, 30, 39.

Datos de Entrada y configuración

39	22	16	14	12	7	30	28	3	10

Resultados de la corrida

Métrica	Valor
Pasadas	2
Colocaciones en buckets (por pasada)	10
Colocaciones totales (estimadas)	20
Tiempo	16 segundos

Ordenamiento Final

3	7	10	12	14	16	22	28	30	39

Comparación

- Ordena por dígitos (LSD o MSD); no compara elementos directamente; puede alcanzar tiempo lineal en n.
- Cómo se compara Frente a Quick/Merge: puede ser más rápido para enteros/strings con dígitos limitados porque evita comparaciones; sin embargo, requiere espacio adicional y depende de d y k.
- Frente a Insertion/Bubble/Selection: supera a estos en escalabilidad cuando los parámetros d,k son pequeños; para N pequeño la ventaja puede ser marginal.
- Frente a Merge: Radix es no comparativo y estable si los buckets lo son; Merge es más general y no depende de representación de claves.
- Recomendación práctica
- Excelente para ordenar enteros o cadenas con longitud limitada y cuando la memoria adicional es aceptable.